

Git e Plataformas: GitHub e GitLab

Controle de versão distribuído e colaboração essencial no desenvolvimento de software.

INSTITUIÇÃO EXECUTORA



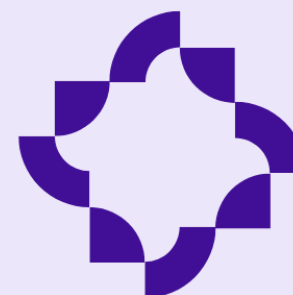
COORDENADORA

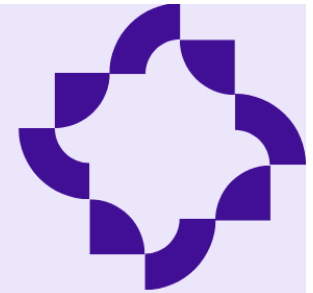


APOIO



DO LADO DO POVO BRASILEIRO





Sumário

- Fundamentos do Controle de Versão e Git
- Áreas do Git e Fluxo de Trabalho Básico
- Ramificação, Convergência e Merge
- Plataformas GitHub e GitLab
- Sincronização Remota e Pull Requests



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Fundamentos do Controle de Versão

O que é VCS?

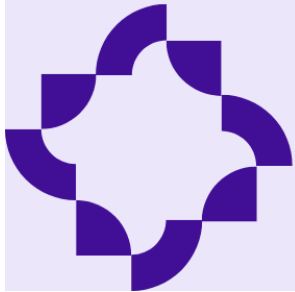
O **Controle de Versão** (VCS) é um sistema que registra as mudanças em um arquivo ou conjunto de arquivos ao longo do tempo, permitindo recuperar versões específicas posteriormente.

Rastrear alterações

Permite **rastreabilidade**, identificando quem alterou o quê e por quê em cada etapa do desenvolvimento.

Recuperar estados anteriores

Possibilita a **recuperação** para estados anteriores do projeto, crucial em caso de *bugs* críticos ou erros.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Fundamentos do Controle de Versão



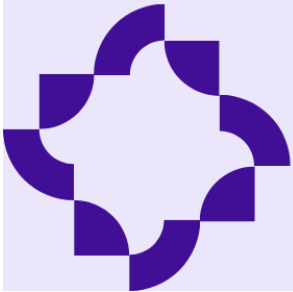
Facilitar colaboração

Essencial para o **trabalho em equipe**, permitindo que múltiplos desenvolvedores colaborem no mesmo código sem sobrescrever o trabalho alheio.



Experimentar com segurança

Oferece um ambiente seguro para **experimentação**, criando ramificações para testar novas funcionalidades sem comprometer a estabilidade do projeto principal.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



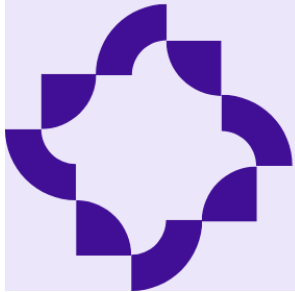


VCS Centralizado

Nesta arquitetura, um **servidor único** armazena todo o histórico do projeto. As operações essenciais como commit, merge ou visualização do histórico dependem diretamente da conexão com este servidor, criando um ponto único de falha. Isso pode impactar negativamente o desempenho em projetos com grandes históricos, devido ao constante tráfego de rede.

VCS Distribuído

Em contraste, cada cliente possui uma **cópia completa** do repositório, incluindo o histórico. Operações como commit e log podem ser realizadas localmente, sem a necessidade de rede, garantindo alta disponibilidade e desempenho. Esta abordagem oferece maior resiliência, pois cada cópia local atua como um backup do projeto completo.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



As Três Áreas do Git

Diretório de Trabalho

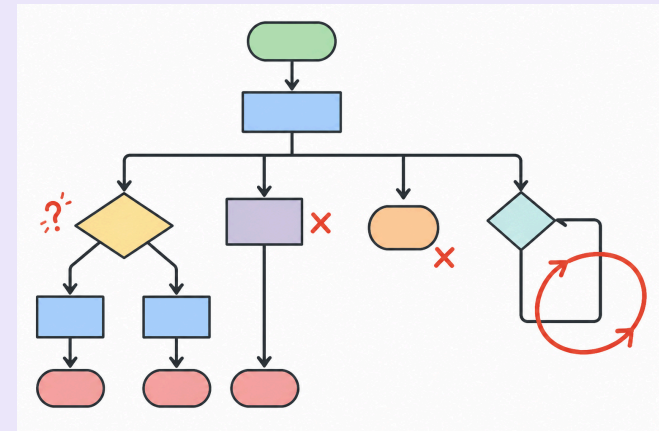
Uma cópia local de uma versão específica do projeto. Os arquivos são *descomprimidos* do diretório Git e disponibilizados no disco para modificação. É onde você interage diretamente com os arquivos do projeto.

Área de Staging

Também conhecida como *Index*, é um arquivo que armazena informações sobre as mudanças que você pretende incluir no próximo commit. Permite *curar* as alterações, selecionando exatamente o que será salvo no histórico do projeto.

Diretório Git

É a parte mais fundamental do Git. Aqui são armazenados todos os metadados do projeto e o banco de dados de objetos (objetos commit, árvore, blob, e tag) que compõem o repositório. Este diretório é copiado quando um repositório é clonado.



Fluxograma: Staging Area, decisões, erros e loops no Git.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Fluxo de Trabalho Básico no Git

1

Modificar Arquivos

Nesta fase inicial, o desenvolvedor realiza alterações nos arquivos do projeto dentro do seu diretório de trabalho. Essas modificações ainda não estão rastreadas pelo Git de forma ativa.

2

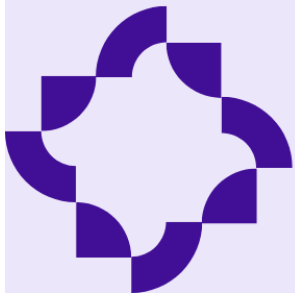
Preparar (Stage) Alterações

Após as modificações, o desenvolvedor seleciona quais alterações específicas farão parte do próximo snapshot, adicionando-as à *área de preparação* (staging area). Este passo é realizado com o comando ``git add``, preparando os arquivos para o commit.

3

Consolidar (Commit) Snapshot

Finalmente, as alterações preparadas são permanentemente armazenadas no repositório local como um novo *commit*. Este comando ``git commit -m "mensagem"`` cria um registro histórico, encapsulando o estado dos arquivos na área de preparação e associando uma mensagem descritiva.



INSTITUIÇÃO EXECUTORA



COORDENADORA



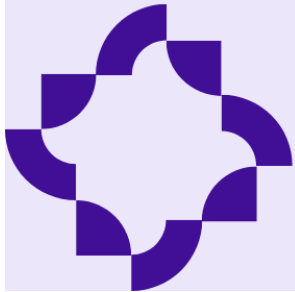
APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Identidade e Configuração no Git

- **`user.name`**: Define o nome do autor que será associado a todos os seus commits globais no Git.
- **`user.email`**: Configura o endereço de e-mail do autor para cada commit, essencial para a identificação no histórico.
- **Configurações Imutáveis**: Após um commit, essas informações são permanentes e parte integral do snapshot gerado pelo Git.
- **`git config --list`**: Use este comando para verificar todas as configurações ativas, incluindo nome e e-mail.



INSTITUIÇÃO EXECUTORA



COORDENADORA



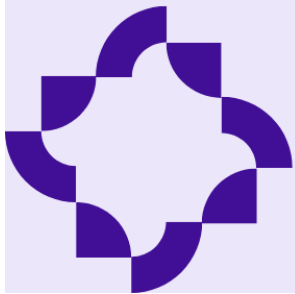
APOIO





A Anatomia de um Commit

- Diferente de outros sistemas que armazenam *deltas* (diferenças), o Git armazena **Snapshots** completos do projeto em cada commit.
- **Hash SHA-1**: Um identificador único de 40 caracteres (ex: 5b3a4...) é gerado para cada commit, assegurando sua integridade e imutabilidade.
- **Ponteiro**: Cada commit contém uma referência direta para o *snapshot* dos arquivos na área de preparação no momento de sua criação.
- **Metadados**: Inclui informações cruciais como o autor da modificação, a data e hora do commit, e uma mensagem descritiva do propósito das mudanças.
- **Parentesco**: O commit armazena ponteiros para seu(s) commit(s) pai(s), estabelecendo a cronologia e a estrutura de árvore do histórico do projeto.



INSTITUIÇÃO EXECUTORA



COORDENADORA



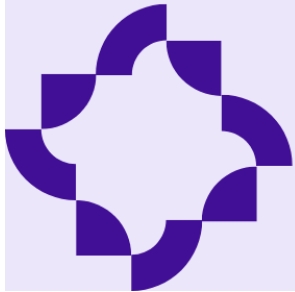
APOIO





Ramificação e Convergência (Branching)

- **Branching** é o recurso mais poderoso do Git, permitindo desvios da linha principal de desenvolvimento de forma isolada.
- A branch 'main' representa a **linha de produção** estável e funcional do projeto, servindo como base.
- No Git, um **branch leve** é um ponteiro móvel para um commit, tornando a criação de novos ramos quase instantânea e eficiente.
- O **isolamento** é crucial, permitindo desenvolver novas funcionalidades ou correções de bugs em ramos separados sem impactar diretamente o código principal.
- Utilize o comando ``git checkout -b `` para criar e alternar rapidamente para um **novo ramo**, facilitando o fluxo de trabalho.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Tipos de Merge

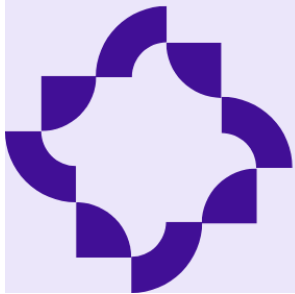
O **Merge** integra as histórias de dois branches, sendo crucial após isolar tarefas. Dois tipos principais ocorrem com base na divergência dos branches:

- **Fast-forward:** Acontece quando o branch alvo não divergiu. O ponteiro do branch alvo é simplesmente movido para o commit mais recente do branch a ser integrado. Não há criação de um novo commit.
- **Three-way Merge:** Ocorre quando ambos os branches divergiram de um ancestral comum. O Git cria um novo 'commit de merge' que sintetiza as mudanças a partir do commit ancestral comum, resultando em uma nova linha no histórico.

Conflitos e Resolução

Durante um merge, podem surgir **conflitos** que requerem intervenção.

- **Causa:** Conflitos emergem quando a mesma linha de um arquivo é alterada de formas diferentes em ambos os branches, ou quando um arquivo foi excluído em um branch e modificado no outro. O Git não consegue decidir automaticamente qual alteração prevalece.
- **Resolução:** O Git pausa o processo de merge e exige que o desenvolvedor resolva manualmente as diferenças. Ferramentas de merge auxiliam na identificação das seções conflitantes, permitindo a edição do código para definir a versão final desejada. Após a resolução, o desenvolvedor marca o conflito como resolvido e finaliza o merge.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Plataformas: GitHub e GitLab

GitHub: Foco Open-Source

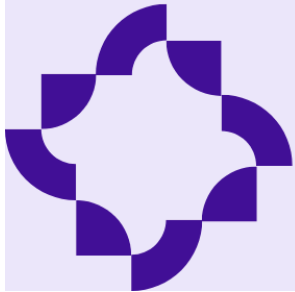
Com um foco robusto na **comunidade open-source**, o GitHub é conhecido por sua vasta biblioteca de projetos e por ferramentas de Integração Contínua/Entrega Contínua (CI/CD) como o *GitHub Actions*. É uma plataforma amplamente utilizada para colaboração e hospedagem de repositórios.

GitLab: DevOps Completo

O GitLab se diferencia por oferecer um **ciclo de vida DevOps completo** (DevSecOps), integrando nativamente funcionalidades como container registry, gerenciamento de projetos e segurança. É frequentemente a escolha preferencial para empresas que buscam uma solução unificada e robusta.

Hospedagem de Repositórios

Ambas as plataformas fornecem a funcionalidade principal de **hospedagem de repositórios remotos**. Elas atuam como uma 'fonte da verdade' centralizada na nuvem, permitindo que as equipes sincronizem seus repositórios Git locais, facilitando a colaboração e a gestão de versões em larga escala.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Sincronização com Remotos

Clonar Repositório Remoto

Baixa um repositório remoto inteiro pela primeira vez para sua máquina local. Utiliza o comando ``git clone`` para iniciar um novo projeto a partir de um servidor remoto.

Buscar Alterações (Fetch)

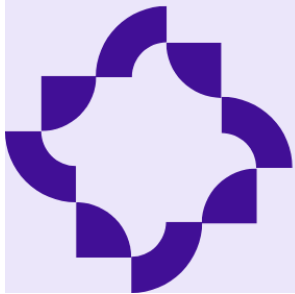
Busca as novidades do servidor remoto (commits, branches), mas não as aplica automaticamente ao seu código local. É uma forma segura de inspecionar as mudanças antes de integrá-las.

Puxar Alterações (Pull)

Combina as operações de ``fetch`` e ``merge``. Baixa as alterações do servidor remoto e tenta integrá-las automaticamente na sua branch local de trabalho. Pode causar conflitos de mesclagem.

Enviar Alterações (Push)

Envia seus commits locais para o servidor remoto, atualizando a branch correspondente no repositório remoto. O comando comum é ``git push origin``, onde 'origin' é o nome do remoto e ' ' é a branch a ser enviada.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Fluxo de Pull Requests

- O **Pull Request (PR)** ou *Merge Request (MR)* é o mecanismo de revisão de código por excelência.
- **Proposta:** Um desenvolvedor propõe a integração de código do seu *branch* pessoal ao repositório principal.
- **Code Review:** Membros da equipe analisam o código, sugerindo melhorias.
- **Gatekeeping:** Permite a execução automática de testes (CI) antes da integração.
- **Governança:** Garante que nenhum código entre em produção sem validação de terceiros.

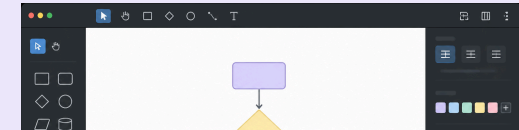
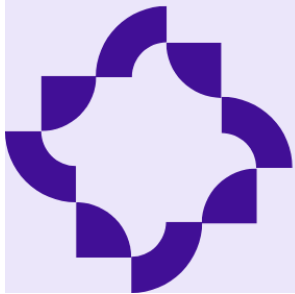


Imagem 2: Interface de software de fluxograma para visualizar fluxos de trabalho de PRs.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

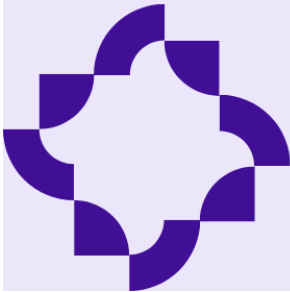




Comparativo de Funcionalidades

Este resumo detalha as capacidades técnicas de cada plataforma para auxiliar na escolha tecnológica de projetos corporativos.

Funcionalidade	GitHub	GitLab
Hospedagem	Nuvem (SaaS)	SaaS e Auto-hospedado
CI/CD	GitHub Actions (YAML)	GitLab CI/CD nativo
Issue Tracking	Simples e intuitivo	Robusto (metodologias ágeis)
Permissões	Baseado em papéis	Muito granular (por branch)
DevOps	Integração via Marketplace	Suíte 'Tudo-em-um'



INSTITUIÇÃO EXECUTORA



COORDENADORA



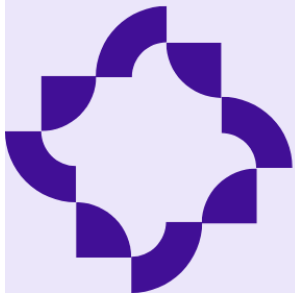
APOIO





Exercício Resolvido

Como demonstrar um fluxo completo de trabalho utilizando Git, desde a criação de uma nova funcionalidade (feature) até a integração remota e a solicitação de revisão (Pull Request) em plataformas como GitHub ou GitLab?



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

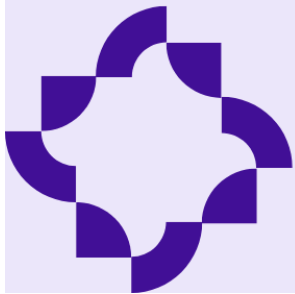
MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Exercício Resolvido

- 1 Crie uma nova branch para a funcionalidade ('feature/login') a partir da branch principal com o comando ``git checkout -b feature/login``, isolando o desenvolvimento da nova funcionalidade do código principal.
- 2 Modifique o arquivo ``auth.js`` com as alterações necessárias e salve, implementando a lógica da nova funcionalidade (ex: autenticação JWT).
- 3 Prepare as alterações para serem incluídas no próximo commit utilizando ``git add auth.js``, selecionando o arquivo específico para o snapshot.



INSTITUIÇÃO EXECUTORA



COORDENADORA



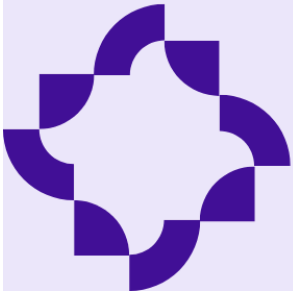
APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Exercício Resolvido

- 4 Registre as alterações em um snapshot no histórico do repositório local com o comando ``git commit -m "Implementa autenticação JWT"``, fornecendo uma mensagem clara que descreva a mudança.
- 5 Envie a nova branch ('feature/login') e suas alterações para o repositório remoto no servidor com ``git push origin feature/login``, tornando o trabalho disponível para outros colaboradores e para revisão.
- 6 Na plataforma remota (GitHub/GitLab), abra um Pull Request (ou Merge Request) da branch ``feature/login`` para a branch ``main``, solicitando que suas alterações sejam revisadas e integradas ao código principal do projeto.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃOGOVERNO DO
DO LADO DO POVO BRASILEIRO



Considerando o fluxo de trabalho demonstrado em ambientes profissionais de desenvolvimento, qual é a ação técnica precisa que o comando ``git pull`` realiza em seu repositório local?

A

Apaga o histórico de commits local e o substitui pelo do repositório remoto.

B

Realiza o download (fetch) das mudanças do repositório remoto e tenta a mesclagem automática (merge) no branch atual.

C

Envia as alterações locais para o repositório remoto, sincronizando o histórico.

D

Apenas baixa os arquivos do repositório remoto sem integrar ao seu branch.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Considerando o fluxo de trabalho demonstrado em ambientes profissionais de desenvolvimento, qual é a ação técnica precisa que o comando `git pull` realiza em seu repositório local?

- ☐ Apaga o histórico de commits local e o substitui pelo do repositório remoto.
- ☒ Realiza o download (fetch) das mudanças do repositório remoto e tenta a mesclagem automática (merge) no branch atual.
- ☐ Envia as alterações locais para o repositório remoto, sincronizando o histórico.
- ☐ Apenas baixa os arquivos do repositório remoto sem integrar ao seu branch.



INSTITUIÇÃO EXECUTORA



COORDENADORA



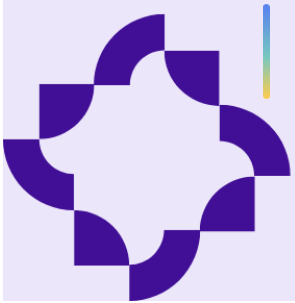
APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Discuta com o colega ao lado

Em um contexto de grandes empresas, qual a vantagem comparativa entre a abordagem Monorepo e Multi-repo para o gerenciamento de código?



Considerem os impactos na velocidade do `git clone`, no gerenciamento de dependências e nos pipelines de CI/CD.

INSTITUIÇÃO EXECUTORA



COORDENADORA

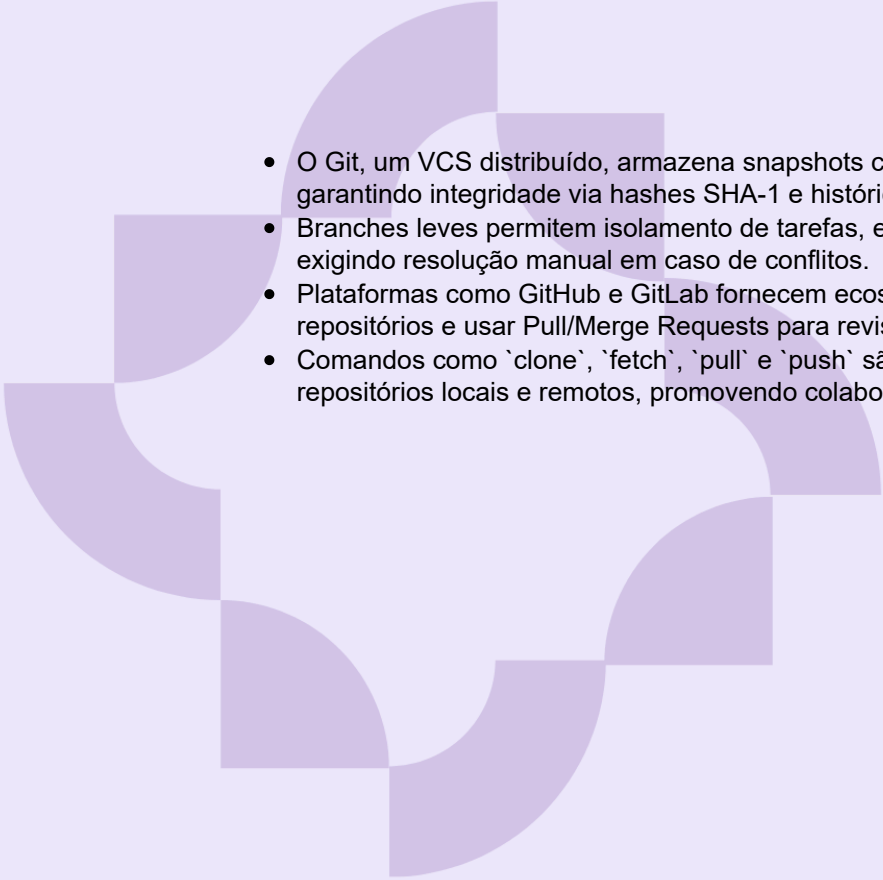


APOIO





Conclusão

- 
- O Git, um VCS distribuído, armazena snapshots completos, não deltas, garantindo integridade via hashes SHA-1 e histórico imutável.
 - Branches leves permitem isolamento de tarefas, e merges unem histórias, exigindo resolução manual em caso de conflitos.
 - Plataformas como GitHub e GitLab fornecem ecossistemas para hospedar repositórios e usar Pull/Merge Requests para revisão de código.
 - Comandos como `clone`, `fetch`, `pull` e `push` são essenciais para sincronizar repositórios locais e remotos, promovendo colaboração eficiente.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO